FORGE: Reverse-Engineering the Reward Economics of a Multi-Step Tool-Attack Benchmark, with a Matched Defense

Christian Metzl
Independent Researcher
christianmetzl@aol.com

August 31, 2026

Abstract

We study the Kaggle *AI Agent Security — Multi-Step Tool Attacks* benchmark, in which an attacker submits candidate conversations that drive two open-weight tool-using models (`gpt-oss-20b`, `gemma-3-4b`) and is scored by the security predicates the models are induced to trigger. Rather than treat the task as prompt engineering, we **reverse-engineer its reward economics** from the evaluation gateway and from a publicly shared competitor solution, and show that a single relation governs everything: score $= 0.09 \times$ (tool calls that fire), a **time-bounded linear knapsack**. The binding constraint is *not* model refusal — single requests comply $\approx 100\%$ — but *posts sustained per candidate* and *candidate throughput* under a fixed replay budget. On this footing we contribute (i) a formal statement of the objective and its optimal allocation; (ii) three throughput levers — aggressive token minimization, bounded multi-message packing, and a *sustain-aware bandit probe* that selects framings by measured value-per-second on the live model; (iii) an honest treatment of "quantum-inspired" optimization, which the linear structure rules out except at the probe; and (iv) a **matched defense** — a prompt-agnostic action-side guardrail plus a chat-template input sanitizer — that collapses our own strongest attacks to zero with no benign false positives. Throughout we tag every claim [measured]/[inferred]/[speculative], and we document our wrong turns — a mis-diagnosed "compliance wall," a regression from "smarter" verbose jailbreaks, and a scoring shortcut we tested and rejected — as the lessons most likely to transfer. This is a defensive study of a sandboxed benchmark with synthetic markers; the one guardrail gap we exploit is disclosed to the organizers with a fix.

1 Introduction

The competition [1] frames a concrete agentic-security question: given only the ability to write the *user side* of a conversation, how reliably can an attacker steer a tool-using LLM into a policy-violating tool call (data exfiltration, an untrusted-content-driven action, a destructive write, a confused-deputy email)? Submissions are code notebooks; a hidden rerun connects an external gateway that (a) lets the attacker *generate* candidates against a live public guardrail, then (b) *replays* the returned candidates against held-out guardrails and both target models, scoring the predicates that fire.

Our system, **FORGE** (Fingerprint-Oriented Replay-Guided Exploration), began as an adaptive multi-route search. Its decisive progress, however, came not from better prompts but from

measuring the scoring function exactly and optimizing the right quantity. This note reports that arc, including the parts where we were wrong.

Contributions.

1. **Reward economics (§2).** We derive, from the gateway source and a competitor solution, the governing relation score = $0.09 \times$ (firing tool calls) and cast the attacker's problem as a time-bounded linear knapsack; we identify *posts-sustained-per-candidate* and *candidate throughput* — not refusal — as the binding constraints.
2. **Throughput levers (§5).** Token minimization, bounded multi-message packing, and a sustain-aware *bandit* probe that adapts framing selection to the live model.
3. **An honest optimization verdict (§6).** The additive, dedup-free scoring makes the allocation a linear knapsack where greedy is optimal; annealing/QAOA/DPP add nothing. The one place a sampling method wins is the probe.
4. **A matched defense (§7).** An action-side, prompt-agnostic guardrail plus a chat-template sanitizer that drive our strongest attacks to zero with zero benign false positives.
5. **Lessons and reproducibility (§9).** Documented self-corrections and a one-command local harness; claims are tagged by evidence class throughout.

2 Benchmark Mechanics and Reward Economics

Scoring ([measured], from `aicomp_sdk/scoring.py`). Let a submission's replay produce a multiset of triggered predicate *events* with severities $s \in \{1, \dots, 5\}$ and severity weights $w(s)$ with $w(5)$=16, $w(4)$=8, $w(3)$=4, and let C be the number of distinct *score cells* (keyed on destination *domain* + tool sequence, not message text). The raw score sums *per event with no de-duplication* plus a novelty bonus, and is normalized:

$$\text{raw} = \sum_{\text{events}} w(s) + 2C, \qquad \text{score} = \min\Big(1000,\ \tfrac{\text{raw}}{200}\Big). \tag{1}$$

The dominant action is an `http.post` carrying a synthetic marker, which fires EXFILTRATION (s=5, 16 raw). With one distinct destination domain per post (a free +2 cell), **each successful post is worth** 18 **raw** = 0.09 **normalized**.

The governing relation. The gateway replays up to 2000 candidates but is bounded by a per-phase wall-clock budget $B \approx 8750$s; each candidate costs $L \approx L_{\text{prefill}} + K\,L_{\text{decode}}$, dominated by a ≈ 25s prefill for a 20B model on the evaluation hardware (a constant a competitor solution encodes directly [2]). Because scoring is additive with no interaction, the score of a submission is simply

$$\boxed{\text{score} \ \approx\ 0.09 \times (\text{tool calls that fire})} \tag{2}$$

and the number of calls that fire is (candidates that fit B) $\times$ (posts sustained per candidate). Crucially, *single-request compliance is* $\approx 100\%$ [2]: refusal is not the wall. The levers are (a) fitting more candidates into B (smaller L) and (b) sustaining more scoring posts per candidate (K_{eff}).

Optimal allocation. With value $v_i = 16K_{\text{eff},i} + 2C_i$ and cost $c_i = L_i$ per candidate route i, maximizing $\sum v_i$ subject to $\sum c_i \le B$ is a *fractional knapsack*; sorting by v_i/c_i and filling greedily is provably optimal. FORGE therefore probes each route on the live guardrail, estimates v_i/c_i from the measured trace, orders best-first, and fills B.

3 FORGE in Brief

FORGE (i) *fingerprints* the live public guardrail during generation by probing multiple framings per route and keeping only those that fire; (ii) ships a *split portfolio* of routes — marker-exfil for the public and taint-style-private columns, untrusted→action for a pattern-style private column, and a whitelisted-domain confused-deputy for the conjunctive worst case — reserving a tunable fraction of the scored prefix for the (public-null) private routes; and (iii) sizes and orders the returned list to the replay budget. Candidate validity (≤ 32 messages, ≤ 2000 chars, non-empty) is enforced so no candidate aborts the run. A local harness reproduces the whole pipeline against the SDK's environment, predicates, scoring, and both packaged guardrails with deterministic stand-in agents.

4 The Throughput Gap, Reverse-Engineered

Our first real-eval submissions scored ≈ 15 (public), while a publicly shared solution reported ≈ 60 and the public leader 147.53. We initially mis-read the gap as a "9% compliance wall." Reading the competitor's own code [2] corrected us:

- single-post framings are "proven $\approx 100\%$ fire" — the 0.09 is the *score of one complying post*, not a compliance *rate*;
- the differentiator is K_{eff}: our same-URL "repeat the post N times" batch sustained ≈ 0.7 of 8 posts, whereas a **distinct-record ETL framing** ("forward each of these N records, one call per record") sustained ≈ 3 — reading as a real batch job rather than a pointless loop;
- longer prompts inflate L_{prefill} and thus *reduce* the candidate count — which independently explains why our "smarter" verbose jailbreaks scored *worse* (§9).

This reframing — from "crack a compliance wall" to "maximize firing posts per second" — is the single most important finding of the study.

5 Innovations Beyond the Public Floor

Treating the competitor mechanism as a *floor*, we add three levers, each targeting a term in Eq. (2).

Token minimization. Short endpoints (`http://d00001.invalid`, ≈ 21 vs 47 chars), minimal distinct records, and a hard output/reasoning suppressor cut message length $\approx 38\%$. Fewer input *and* output tokens shrink both L_{prefill} and L_{decode}, raising both candidate count and K_{eff}.

Bounded multi-message packing. A single interaction caps at 8 tool hops; exceeding 8 posts/candidate requires multiple messages, which *crashed* at high density (context-length OOM on a 20B CPU model). We bound it to 2–3 ultra-short messages (16–24 posts/candidate) so the accumulated context stays $\approx 8\times$ smaller than the build that crashed, amortizing one 25s prefill over many posts.

Sustain-aware bandit probe. The naive probe commits to the *first* framing that fires; "fires once" $\neq$ "sustains the most." We instead treat framing choice as a budget-constrained multi-armed bandit: draw each of a few framings r times, estimate its v/c, and commit to the best. This adapts K_{eff} to the *actual* rerun model, which is unobservable offline — an edge no static framing has.

A lever model calibrated on the competitor's constants suggests these compound from the ≈ 58 floor toward ≈ 100 (token-min) and, with working packing, well beyond ([inferred]; the projections are not measured).

6 On "Quantum-Inspired" Optimization

Because scoring is additive with no dedup and distinct domains yield distinct cells, there are no diminishing returns or cross-candidate interactions: the allocation is a *linear* knapsack, for which greedy is provably optimal. Determinantal point processes, simulated/quantum annealing, and QAOA offer nothing over greedy here — indeed our DPP selector reduces exactly to greedy and is disabled. The one sub-problem with genuine stochastic structure is *which framing to commit to under a tiny query budget against a noisy model*: a multi-armed bandit, solved in Thompson-sampling spirit by the sustain-aware probe. The principled "sampling-based" optimizer and the real algorithmic edge are thus the same object; we resist dressing greedy as something exotic.

7 From Attack to Defense

The public guardrail has one genuine gap (*Bug 1*, [source-traced]): it inspects an `http.post`'s *URL* for danger but not its *data* channel, so a benign-looking URL carrying the marker in `data` is allowed and scores. We report it here and disclose it to the organizers with a fix. Our matched defense has two layers:

1. **Action-side, prompt-agnostic inspection.** A guardrail that checks the `data` channel with the scorer's own matcher and tracks provenance (secret-read taint, untrusted-source taint) denies the exfil *regardless of framing*. Every compliance jailbreak we tried — role-play, Crescendo, many-shot, "fake over-refusal," policy-collision — changes the model's willingness but not the tool call, and so cannot pass this boundary.

2. **Input-side sanitization.** The one attack a pure action-filter cannot pre-empt is chat-template *forgery* (injecting Harmony/`gemma` control tokens to forge a prior turn); a small sanitizer neutralizes those tokens in untrusted content before templating.

Measured: running our strongest attack builds against the reference guardrail collapses their score to 0 (e.g. normalized $23.8 \to 0$), with *zero* benign false positives. Stopping a jailbreak at the action boundary beats an input classifier the jailbreak is engineered to slip past — the transferable defensive lesson.

8 Results (Measured, Real Evaluation)

All numbers below are from the live competition evaluator (public column, $\sim$0–1000 scale).

- **Throughput vs. single-post.** A terse batch-8 build scored **14.915**, +36% over the best single-post build (**10.935**) at the same configuration — consistent with Eq. (2) (batch amortizes the fixed prefix).

- **The private-coverage lever is clean and linear.** Sweeping the fraction of the scored prefix reserved for public-null private routes gives public $\{0.05 : 10.94,\ 0.15 : 8.86,\ 0.30 : 7.69,\ 0.45 : 5.74,\ 0.60 : 4.29\}$, a near-linear public $\approx 11.5 - 12.1 \cdot \text{pf}$: each unit of private coverage trades public-scoring posts as modeled.
- **Verbose "jailbreaks" regressed.** Adding role-play / Crescendo / many-shot framings *lowered* the score ($10.94 \to 7.58$ and 7.47) — longer prefixes cost candidates and mangle the tool JSON on small models. Terse wins twice.
- **Multi-message dense crashed** on the real eval (Kaggle Error), motivating the bounded, token-minimized hybrid.

The private column that decides the final rank is hidden until close; we therefore hedge a dedicated private-coverage submission and treat the public projections above ≈ 15 as [inferred], not established.

9 Lessons Learned

1. **Measure the objective before optimizing it.** Weeks of "compliance" effort dissolved once we read the scorer: the constraint was throughput, not refusal.
2. **A wall can be a mis-read.** The "9%" was posts-sustained, not a refusal rate; the fix was candidate structure (distinct-record ETL), not a better jailbreak.
3. **Cleverness can cost points.** Literature-grade jailbreaks (role-play $\sim$71% on `gpt-oss` *for chat content*) *regressed* tool-call throughput — their length hurt both candidate count and JSON fidelity. Negative results matter.
4. **Test the shortcut before believing it.** A "stack two predicates per post" ($16 \to 32$) idea was killed by a one-line experiment: the public guardrail taint-blocks the post after an untrusted read, so both predicates die.
5. **Attack and defense are one project.** The prompt-agnostic guardrail that zeroes our attack is the same artifact that makes the finding useful to defenders.

10 Responsible Disclosure and Ethics

This is defensive research on a *sandboxed* benchmark: the "secret" is a synthetic literal (`SECRET_MARKER`), destinations are reserved `*.invalid`/`*.example` names, and no real system or person is targeted — the competition's stated purpose. The single guardrail gap we exploit (Bug 1) is documented and will be disclosed to the organizers [1], together with the fixed reference guardrail and input sanitizer in the accompanying code. We publish no operational capability against any real deployment.

Reproducibility

The attack, the local evaluation harness, the reference defense, the claims ledger (with every claim tagged by evidence class), and the build presets discussed here are released with this note. A single command reproduces the mechanism checks against the SDK's own environment, predicates, and guardrails.

Acknowledgments

Development, analysis, and drafting were carried out with the assistance of Claude Code (Anthropic). We thank the organizers for a benchmark whose *scoring function itself* rewarded careful reading, and the competitors whose public notebooks made the reward economics legible.

References

[1] M. Bhatt, C. Huang, O. Vallis, J. Chang, S. Mathews, B. Gatto, M. Cruz, Y. Yan, and M. Plomecka. *AI Agent Security — Multi-Step Tool Attacks*. Kaggle, 2026. `https://kaggle.com/competitions/ai-agent-security-multi-step-tool-attacks`.

[2] yusuketogashi. *lb60-525-july-safe-edge-prune-tail8-upgrade* (public competition notebook; portfolio/latency-sizing lineage from pilkwang). Kaggle, 2026. `https://www.kaggle.com/code/yusuketogashi/lb60-525-july-safe-edge-prune-tail8-upgrade`.

[3] pilkwang. *AI-Agent single-post exfiltration; replay dense exfiltration* (public competition notebooks). Kaggle, 2026. `https://www.kaggle.com/pilkwang`.

[4] nctuan. *JED slow multipost* (public competition notebook). Kaggle, 2026. `https://www.kaggle.com/code/nctuan/jed-slow-multipost`.

[5] C. Anil et al. *Many-shot Jailbreaking*. Anthropic, 2024.

[6] Z. Chen et al. *Bag of Tricks for Subverting Reasoning-Based Safety Guardrails*. arXiv:2510.11570, 2025.

[7] *Dialogue Injection Attack: Jailbreaking LLMs through Context Manipulation*. arXiv:2503.08195, 2025.

[8] *Quant Fever, Schrödinger's Compliance, . . . : Probing GPT-OSS-20B*. arXiv:2509.23882, 2025.

[9] *Sequential Tool-Attack Chaining (STAC)*. arXiv:2509.25624, 2025.